CE807 – Assignment

ZEINAB DAST MOZD (2323135)

DATA

- I received a comprehensive dataset containing customer opinions about purchased products, structured into training, validation, and test sets. The training and validation datasets include two key columns: one containing the text of customer opinions and the other with labels indicating whether the sentiment expressed is positive or negative.

Data set	Total	Class A: POSITIVE	Class B: NEGATIVE
Train	3681	3030	651
Valid	454	374	80
Test	409	NaN	NaN

Model’s selection

SVM

- provides clear decision boundaries, making it well-suited for distinguishing between positive and negative sentiments with high accuracy
- robust performance in text classification tasks, particularly when dealing with high-dimensional data such as word vectors.
- SVM works well with various types of data, including text, and can be applied to both linear and non-linear classification problems using kernel functions.

K MEANS

- doesn't require labelled data, straightforward to classify and group similar data points
- efficiency in handling large-scale datasets, making it ideal for initial data exploration and reducing the complexity of data
- K-means is also straightforward to implement, with a simple algorithmic process that makes it accessible and easy to use for various data analysis tasks

FCNN

- FCNNs have been extensively studied and are well-supported in many deep learning frameworks like TensorFlow, PyTorch, and Keras. This maturity means there is a wealth of resources, tutorials, and pre-trained models available.

Text Cleaning

- Lowercasing**:** Converted all text to lowercase so that words like "Happy" and "happy" are treated the same.
- Removing Punctuation and Digits: Eliminated punctuation marks and numbers, which don’t contribute much to the sentiment analysis in my opinion.
- Stop word Removal: Removed common words like "the," "is," and "and" that don’t have significant meaning in sentiment analysis.
- Tokenization: Split the text into individual words or tokens for more detailed analysis.
- Stemming/Lemmatization: Reduced words to their base forms, such as changing "running" to "run," to focus on the core meaning of the words.

Additional cleaning method

- I've explored various additional methods for cleaning text, such as removing special formatting tokens, eliminating markers, replacing multiple spaces with a single space, and filtering out URLs and email addresses. Despite implementing these techniques, they didn't yield any noticeable improvement in the quality or performance of my text processing.
- Therefore, I've decided to focus on the essential and sufficient cleaning methods. By concentrating on these core techniques, I ensure that the text is clean and consistent without overcomplicating the process. This streamlined approach allows for a more efficient and effective handling of the text data, maintaining its integrity while still being practical and time-efficient.

Vectorization

- In text preprocessing pipeline, I utilized the TF-IDF Vectorizer (Term Frequency-Inverse Document Frequency) to transform the cleaned text into numerical features. This technique helps us capture the importance of words within individual documents relative to the entire corpus.
- TF-IDF is a popular and effective method widely used in natural language processing. It assigns higher values to words that appear frequently in a document but are less common across all documents, thus highlighting significant terms while reducing the weight of commonly used words.
- The numerical vectors generated by TF-IDF are highly compatible with various machine learning algorithms. In my assignment, I employed these vectors in conjunction with models such as Support Vector Machines (SVM) and k-means clustering. Additionally, I extended my analysis by incorporating deep learning models, ensuring a comprehensive comparison across different methodologies.
- By using TF-IDF as a consistent feature extraction method across all chosen models, I maintained a standardized basis for evaluating their performance. This uniformity allowed us to focus on the intrinsic strengths and weaknesses of each model, providing clear insights into their effectiveness in processing and understanding text data.

Modelling Analysis and discussion

- To analyze the sentiment of customer opinions, I applied K-Means clustering (k=2) for unsupervised learning, grouping text into positive and negative sentiments. For classification, I utilized Support Vector Machines (SVM) with optimal parameters found through grid search and cross-validation, achieving the best accuracy among tested models. Despite exploring additional neural network models like CNN and FCNN, SVM proved most effective and efficient. Notably, our dataset exhibited an imbalance with more positive than negative examples, challenging the model in accurately classifying negative sentiments.

Modelling Analysis and discussion

- I obtain the models results and it is shown in table below. From the table we can understand that SVM is outperform all previous models it is even better than neural network model. Moreover, the worst performance was achieved by k-means model.

model	F1 score
SVM	0.849
k-means	0.706
CNN	0.847
FCNN	0.842

Modelling Analysis and discussion

- The results in the Table below show that both SVM and K-means predict more positive sentiments, reflecting the dataset's imbalance. K-means, though slightly more balanced, still leans toward positive predictions, with an overestimation of negative sentiment in the validation set. To better assess performance, we should examine true positive, true negative, false positive, and false negative rates, as high false rates can indicate lower accuracy despite apparent balance. Both methods' positive sentiment bias suggests the need for techniques like resampling or weighting to address the imbalance.

Data set	Svm	k-means
Train positive	3178	3387
Train negative	503	294
Valid positive	417	374
Valid negative	37	80
Test positive	379	378
Test negative	30	31

Justification of model’s performance

- The performance difference between K-means and the other models (SVM, CNN, FCNN) stems from several factors. K-means, a clustering algorithm, doesn't explicitly learn sentiment patterns, leading to less effective sentiment classification. Although the same feature representation was used across models, SVM, CNN, and FCNN, being supervised, leverage labelled data to better distinguish sentiments. K-means' unsupervised nature and lack of sentiment label utilization also contribute to its lower performance. Additionally, advanced models like CNNs and FCNNs capture contextual nuances more effectively but are computationally intensive, making SVM the best balance of performance and efficiency.

Summary

- In conclusion, I analyzed product reviews to classify sentiments as positive or negative. After cleaning and preparing the data, I found that supervised models like Support Vector Machines (SVM) outperformed unsupervised models such as K-means, particularly in detecting negative sentiments. The results emphasize the value of labeled data for effective sentiment analysis, with more experimentation needed to refine the approach and improve accuracy further.

Lessons learned

- This assignment taught me that learning and improving in sentiment analysis requires time and experience, emphasizing the value of experimenting with different approaches. Simple models can be surprisingly effective and efficient. I also learned that handling imbalanced data is crucial, as it can significantly impact model performance. The experience underscored the importance of thorough text preprocessing, thoughtful feature engineering, and careful selection of models and evaluation metrics to achieve accurate sentiment classification.